Ejecutar y depurar Python en VS Code

Terminal, selección de intérprete, launch.json, breakpoints y depuración paso a paso

Objetivo de esta nota

Usa esta guía cada vez que quieras ejecutar un archivo .py desde la terminal de VS Code o inspeccionar tu programa paso a paso con el depurador. Está pensada para el workspace Python_Engineering del curso.

Modelo mental

```
ESCRIBIR CÓDIGO  ->  EJECUTAR EN TERMINAL  ->  OBSERVAR SALIDA
                                    |
                                    v
                              PONER BREAKPOINTS
                                    |
                                    v
                              INICIAR DEBUGGER
                                    |
                                    v
                    INSPECCIONAR / AVANZAR / CORREGIR
```

Dos acciones distintas

Paso	Qué hacer	Por qué
RUN	Ejecutar el programa normalmente.	Ideal para obtener la salida final o ver un error.
DEBUG	Ejecutarlo bajo el depurador.	Ideal para observar cómo cambian variables y flujo línea a línea.

Importante

Un script Python sencillo puede depurarse en VS Code sin launch.json: pulsa F5 y elige Python File. Creamos launch.json porque guarda una configuración explícita y reutilizable.

1. Ejecutar un programa Python desde la terminal

Al abrir Python_Engineering como workspace, la terminal integrada normalmente comienza en la raíz del proyecto. Esto hace predecibles las rutas y después simplifica imports, ficheros y debugging.

Abrir la terminal

Paso	Qué hacer	Por qué
1	Abre Python_Engineering en VS Code.	Será la raíz del workspace.
2	Abre Terminal > New Terminal o pulsa Ctrl+`.	VS Code abre la terminal integrada.
3	Comprueba la ruta del prompt.	Debería apuntar a Python_Engineering.

Comprobar Python

En Windows prueba primero:

```
python --version
```

Si no se reconoce, prueba el launcher:

```
py --version
```

Ejecutar el ejercicio actual

Opción A - entrar en la carpeta de ejercicios:

```
cd .\01_python_bootcamp\week_01_logic\class_01_equation_to_algorithm\exercises\npython ex01_dynamic_pressure.py
```

Opción B - ejecutar la ruta desde la raíz:

```
python .\01_python_bootcamp\week_01_logic\class_01_equation_to_algorithm\exercises\ex01_dynamic_pressure.py
```

Recomendación mientras aprendes

Usa bastante la Opción A al principio. Navegar con cd te ayudará a entender desde dónde se está ejecutando un programa; luego será fundamental para rutas relativas.

Comandos útiles en Windows / PowerShell

Paso	Qué hacer	Por qué
pwd	Muestra la carpeta actual.	Comprueba el working directory.
ls / dir	Lista archivos y carpetas.	Permite ver qué tienes disponible.

Paso	Qué hacer	Por qué
cd carpeta	Entra en una carpeta.	Cambia el working directory.
cd ..	Sube un nivel.	Facilita navegar el árbol.
cls	Limpia la terminal.	Mantiene la consola legible.

2. Asegúrate de usar el intérprete correcto

La terminal y el depurador deberían utilizar el entorno Python previsto. Esto será cada vez más importante cuando instalemos NumPy, SciPy, pandas o dependencias específicas.

Seleccionar intérprete

Paso	Qué hacer	Por qué
1	Pulsa Ctrl+Shift+P.	Abre Command Palette.
2	Busca Python: Select Interpreter.	Muestra los entornos detectados.
3	Selecciona el entorno del curso.	VS Code lo asocia al workspace.
4	Abre una terminal nueva si hace falta.	Refresca la activación.

Comprobación útil

Ejecuta python --version y, cuando usemos paquetes, python -m pip list. Si el debugger y la terminal se comportan distinto, revisa primero el intérprete seleccionado.

Ejecutar sin escribir el comando

La extensión de Python ofrece Run Python File in Terminal arriba a la derecha. Es cómodo, pero quiero que también aprendas a usar de forma explícita:

```
python tu_script.py
```

Ejecutar archivo vs. REPL

Si escribes solamente python, normalmente entras en el intérprete interactivo identificado por >>>. Para ejecutar un script guardado debes incluir su nombre.

```
python
>>> 2 + 2
4
>>> exit()

python ex01_dynamic_pressure.py
```

3. Crear launch.json para depurar Python

VS Code guarda las configuraciones del depurador en launch.json, dentro de una carpeta .vscode del workspace:

```
Python_Engineering\n└── .vscode\n    └── launch.json
```

Generarlo desde VS Code

Paso	Qué hacer	Por qué
1	Abre Run and Debug con Ctrl+Shift+D.	Es el panel principal de depuración.
2	Selecciona create a launch.json file o Run > Open Configurations.	Inicia la creación de configuración.
3	Selecciona Python Debugger.	Elige el depurador actual.
4	Selecciona Python File.	Crea configuración para un script normal.
5	Guarda el archivo generado.	Ya queda reutilizable.

Configuración recomendada para el curso

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Python Debugger: Current File",
            "type": "debugpy",
            "request": "launch",
            "program": "${file}",
            "console": "integratedTerminal",
            "cwd": "${workspaceFolder}"
        }
    ]
}
```

No uses el tipo antiguo

La depuración actual de Python usa "type": "debugpy". Si encuentras tutoriales antiguos con "type": "python", cámbialo.

4. Entender qué dice launch.json

No quiero que los archivos de configuración parezcan magia. Léelos como instrucciones para VS Code.

Paso	Qué hacer	Por qué
version	Versión del esquema de configuración.	Normalmente no la toques.
name	Nombre mostrado en Run and Debug.	Puedes cambiarlo.
type	Implementación del depurador: debugpy.	Es la opción actual para Python.
request	launch inicia un proceso nuevo.	Más adelante veremos attach.
program	${file} significa el archivo abierto.	Ideal para ejercicios.
console	integratedTerminal dirige I/O a la terminal.	Útil con print() e input().
cwd	${workspaceFolder} fija Python_Engineering como carpeta de trabajo.	Hace predecibles las rutas relativas.

Por qué importa cwd

Si un programa futuro lee data/input.csv, la ruta relativa se interpreta desde el working directory. Fijar el workspace como cwd hace el comportamiento más reproducible.

```
"cwd": "${workspaceFolder}"
```

Añadir argumentos más adelante

Cuando construyamos scripts con parámetros de entrada, launch.json podrá pasarlos automáticamente:

```
"args": [
    "--altitude", "10000",
    "--mach", "0.8"
]
```

No lo necesitas todavía; sirve para ver por qué una configuración guardada resulta útil cuando el proyecto crece.

5. Depurar paso a paso

El debugger no sirve solo para arreglar errores. Es una de las mejores herramientas para aprender lógica porque permite observar cómo evoluciona el estado del algoritmo.

Programa de ejemplo

```
rho = 1.225
velocity = 100.0

dynamic_pressure = 0.5 * rho * velocity**2

print(dynamic_pressure)
```

Poner un breakpoint

Haz clic en el margen izquierdo de un número de línea. Aparecerá un punto rojo. Cuando la ejecución alcance esa línea, VS Code se detendrá antes de ejecutarla.

Controles principales

- F5 - Continue / Start Debugging: ejecuta hasta el siguiente breakpoint o hasta el final.
- F10 - Step Over: ejecuta la línea actual sin entrar en funciones llamadas.
- F11 - Step Into: entra dentro de una función.
- Shift+F11 - Step Out: termina la función actual y vuelve al llamador.
- Shift+F5 - Stop: termina la sesión.

Qué inspeccionar

Paso	Qué hacer	Por qué
Variables	Valores locales y globales actuales.	Ves cómo cambia el estado.
Watch	Expresiones que quieres vigilar.	Útil para b**2 - 4*a*c.
Call Stack	Funciones que llevaron a la línea actual.	Será clave con módulos.
Debug Console	Evalúa expresiones mientras está pausado.	Prueba valores sin editar el script.

6. Flujo recomendado para los ejercicios de Clase 01

En los cuatro primeros ejercicios usa el debugger deliberadamente. Queremos conectar razonamiento matemático con estado del programa.

Paso	Qué hacer	Por qué
1	Escribe comentarios INPUTS / OUTPUTS / EQUATIONS / SPECIAL CASES / ALGORITHM.	Pensar antes de codificar.
2	Escribe la implementación en Python.	Traducir el algoritmo.
3	Ejecuta desde terminal.	Comprobar comportamiento normal.
4	Pon un breakpoint antes de la ecuación principal o primer if.	Elegir un punto de observación.
5	Pulsa F5 y avanza con F10.	Ver el orden real de ejecución.
6	Inspecciona variables tras cada línea importante.	Relacionar código y estado matemático.
7	Prueba una entrada poco habitual.	Comprender los casos límite.

Ejemplo: ecuación cuadrática

Coloca un breakpoint justo después de calcular el discriminante:

```
discriminant = b**2 - 4*a*c

if discriminant > 0:
    ...
elif discriminant == 0:
    ...
else:
    ...
```

Después prueba tres conjuntos de entrada para recorrer las tres ramas: dos raíces reales, raíz doble y ninguna raíz real.

Problemas comunes

Paso	Qué hacer	Por qué
python no se reconoce	Python no está en PATH o se usa otro launcher.	Prueba py --version y revisa intérprete.

Paso	Qué hacer	Por qué
Paquetes/intérprete incorrectos	Terminal y debugger usan entornos distintos.	Python: Select Interpreter.
Breakpoint ignorado	La línea quizá no es ejecutable o se ejecuta otro archivo.	Comprueba archivo y configuración.
Error launch.json	Tutoriales antiguos usan type: python.	Usa type: debugpy.
File not found	El working directory no coincide con la ruta relativa.	Mira pwd y usa cwd cuando convenga.

7. Chuleta de una página

Ejecutar desde terminal

```
# Abrir terminal
Ctrl+`

# Comprobar Python
python --version

# Navegar
cd .\01_python_bootcamp\week_01_logic\class_01_equation_to_algorithm\exercises

# Ejecutar script
python ex01_dynamic_pressure.py
```

Crear configuración de depuración

```
Ctrl+Shift+D
-> create a launch.json file
-> Python Debugger
-> Python File
```

launch.json mínimo

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Python Debugger: Current File",
      "type": "debugpy",
      "request": "launch",
      "program": "${file}",
      "console": "integratedTerminal",
      "cwd": "${workspaceFolder}"
    }
  ]
}
```

Teclas de depuración

Paso	Qué hacer	Por qué
F5	Iniciar / continuar	Ejecuta hasta breakpoint o final.
F10	Step Over	Ejecuta la línea actual.
F11	Step Into	Entra en una función.
Shift +F11	Step Out	Sale de la función actual.
Shift +F5	Stop	Termina la depuración.

Hábito del curso

Cuando un resultado te sorprenda, no llenes inmediatamente el código de print(). Pregúntate: ¿en qué línea deja el estado del programa de coincidir con lo que espero matemáticamente? Pon ahí un breakpoint e inspecciona el estado.

Base de referencia: documentación oficial de Visual Studio Code sobre terminal integrada, ejecución de Python y depuración con debugpy (comprobada en agosto de 2026).